Building Agentic Applications with LangChain and Deep Agents

These notes follow the demo code in the `main` branch.
`src/01–04` are adapted from the LangChain Foundation course;
`src/05–09` from the Deep Agents course.
This is a curated subset, not a full copy of either course.

1. Requirements

Purpose: Make sure your environment is ready before we start.

Requirement	Why you need it
Python 3.12+	The demos use modern Python syntax and are tested on 3.12
uv	Installs dependencies and runs notebooks and scripts reproducibly
Git	Clones the repository and switches between branches
An API key	At least one provider: OpenAI, Anthropic, Google, or OpenRouter. Ollama is optional for local models
Internet access	Provider APIs and the LangChain docs MCP server
Optional: LangSmith key	Tracing and debugging, if you want to follow the agent's steps

The Python dependencies are declared in `pyproject.toml` and installed with `uv sync`. They include LangChain, LangGraph, Deep Agents, the MCP integration, QuickJS, Pydantic Settings, and Questionary.

```
git clone https://github.com/kshku/LangChain-Workshop.git
cd LangChain-Workshop
uv sync
cp .env.example .env
```

Then open `.env` and fill in `MODEL_PROVIDER`, `MODEL_NAME`, and the API key for your provider. The `chinook.db` file used by the interpreter demo is already in the repository.

To rebuild these notes you also need Typst and Graphviz, but that is not required to follow the session.

2. Introduction

Purpose: Interactive beginning.

2.1. Discussion

- What are the chatbots you used?
- How long you have been using?
- Have you noticed how these applications changed over last few years?
- What can today's AI assistants do that earlier chatbots couldn't?

Earlier chatbots	Modern AI assistants
Primarily text-based conversations	Multimodal interactions

Earlier chatbots	Modern AI assistants
Respond to individual prompts	Handle multi-step tasks
Limited external capabilities	Can interact with external tools
Mostly stateless interactions	Can maintain conversational context
Generate responses	Can take actions on behalf of users

Can AI perform actions for us instead of just answering?

3. What is an Agent?

Purpose: Introduce to agentic systems.

An agent is an AI system that can reason about a task, decide what actions to take, interact with its environment, and work towards achieving a specified goal.

What makes an application agentic? An application becomes agentic when it gives the AI system the ability to determine and execute actions towards a goal.
Examples:

- Reading emails and identifying those requiring attention.
- Writing and executing code, then examining the execution output.
- Searching multiple sources and synthesizing information.
- Interacting with external services through tools and APIs.

The system can observe the results of its actions and use that information to decide what to do next.

4. Chat models and Providers

Purpose: Introduce chat models, model providers and how applications interact with them.

According to what we understood till now, we know we need an intelligent system to do the reasoning and make decisions. We call it the brain of the Agent. Usually we use chat models as the brain of the Agent.

What is the difference between chat models and LLMs?

4.1. Chat Models

There are several Large Language Models (LLMs) available today, such as GPT, Claude, Gemini, Grok, DeepSeek, etc. These models are trained on large amounts of data and can understand and generate human-like text. Chat models are models designed to interact through conversational messages.

Building and training our own LLMs from scratch requires enormous amounts of data, computational resources and infrastructure. This is generally beyond the scope of our application development. Instead, we use already-trained models made available by different providers.

4.2. Model Providers

A model provider is an organization or service that makes models available for applications to use.

Examples:

- OpenAI – GPT models

- Anthropic – Claude models
- Google – Gemini models
- DeepSeek – DeepSeek models
- xAI - Grok models

These providers expose APIs that allow external applications to interact with their models. Each provider may define its own API, authentication mechanism, request format and response format.

For example, when we want to use a model:
1. We obtain an API key from the provider.
2. Our application sends a request to the provider's API.
3. The provider processes the request using the selected model.
4. The model generates a response.
5. The provider returns the response to our application.

Example API references:
- OpenAI API Reference
- Gemini API Reference

Imagine writing an app using OpenAI's api, and later you want to shift to Gemini. Now you need to understand Gemini's API, authentication mechanisms, request and response formats, and we may also need to rewrite code!

What if we had a common interface which deals with different models regardless of the provider API?

This is one of the problems that LangChain helps to solve.

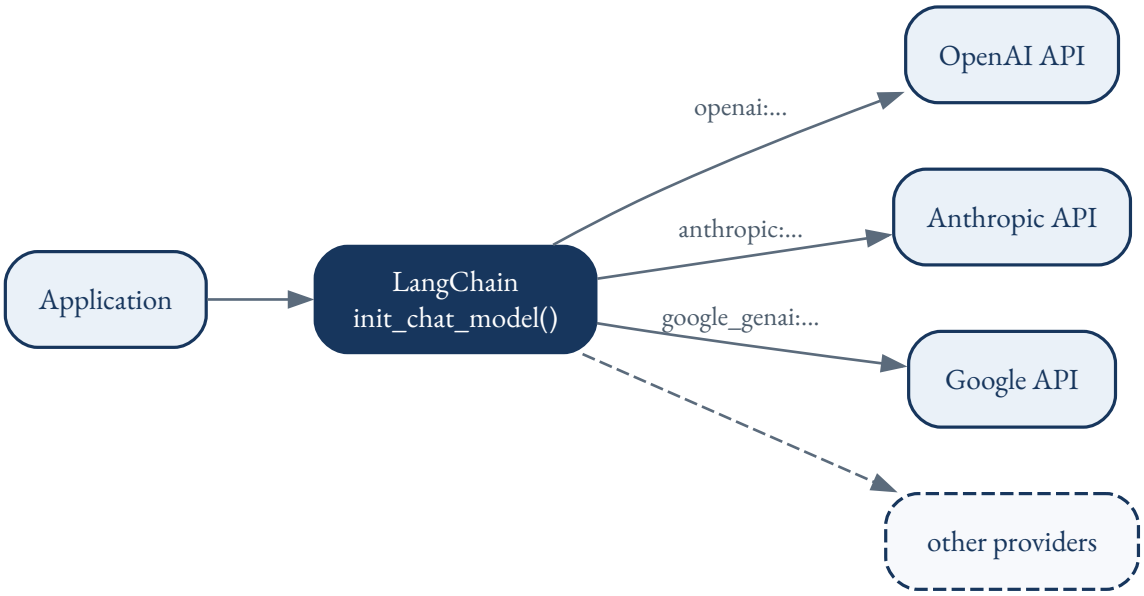


Figure 1: One interface in the application, provider-specific APIs underneath.

5. What is LangChain

Purpose: Introduce to LangChain

LangChain (github link) is an open-source framework and ecosystem for developing applications powered by Large Language Models (LLMs). It provides abstractions and integrations that simplify working with models, tools, data sources and agentic workflows.

Component	What it provides
LangChain	High-level abstractions for building LLM applications and agents
LangGraph	Graph-based orchestration for stateful and long-running agent workflows
Deep Agents	A higher-level agent harness built on LangGraph
LangSmith	Observability, evaluation and deployment capabilities

6. Foundation

Purpose: Get familiar with initializing a chat model, agent and interacting with them using LangChain.

6.1. Project Setup

Setup `MODEL_PROVIDER`, `MODEL_NAME`, and the corresponding provider API key in the `.env` file.

An `.env.example` file is already provided. Copy it to `.env` and fill in the values for the provider you want to use.

Install the dependencies with `uv sync`, then start the notebooks with `uv run jupyter lab` from the repository root.

The `main` branch contains two shared modules:

- `settings.py` -> Loads application configuration using Pydantic Settings and loads environment variables using `python-dotenv`.
- `models.py` -> Initializes the chat model used throughout the demo code, except for the first model-connection demo.

`settings.py` is where the application reads `MODEL_PROVIDER` and `MODEL_NAME` from the environment. Because the fields are required, the application fails early with a clear error if the `.env` file is incomplete. `models.py` builds the shared `model` object once, so the demos do not repeat the same setup code. It also exposes `summarization_model`, which we will use later when summarizing long conversations.

6.2. Connecting to a chat model

The first demo connects to a model directly, without the shared modules, so we can see the raw interaction.

`init_chat_model` takes a model identifier in the form `provider:model`. For example, `openrouter:openai/gpt-oss-120b` selects the OpenRouter provider and an OpenAI model served through it.

```
from langchain.chat_models import init_chat_model

model = init_chat_model(model="openrouter:openai/gpt-oss-120b")
response = model.invoke("Hi, there! How are you!")
```

`response.content` contains the generated text. `response.response_metadata` contains provider-specific details such as token usage, finish reason, or the model that actually served the request. It is worth looking at both, because the metadata often explains *why* a response ended the way it did.

Try different providers and different models.

For supported providers, changing the provider and model configuration is usually enough. The corresponding LangChain integration must be installed, and LangChain handles the provider-specific interaction underneath.

You will frequently refer to the Documentation of LangChain throughout the session.

Try out parameters such as `temperature`, `max_tokens`, `timeout`, and `max_retries`. Observe how changing these parameters affects the model interaction.

Demo code: `src/01-foundation/connecting-to-model.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

6.3. Creating Agent

An agent wraps the model in a loop. It receives messages, decides whether to answer or call a tool, runs any requested tools, and repeats until it has a final answer.

```
from langchain.agents import create_agent

agent = create_agent(model=model)
```

With no tools attached, the loop is short – but it is the same loop that later drives all of our demos.

We invoke the agent with a list of messages:

```
from langchain.messages import HumanMessage

response = agent.invoke(
    {"messages": [HumanMessage(content="What is the capital of the Moon?")]}
)
print(response["messages"][-1].content)
```

The response contains the whole conversation, not just the final text. That is useful: we can inspect what the agent received, what it decided, and what it sent back.

Messages have roles. `HumanMessage` is the user, `AIMessage` is the model, and `ToolMessage` will appear later when the agent uses a tool. You can also pass messages as tuples such as `("human", "...")` or as dictionaries with a `role` and `content`.

A useful experiment is to pass a short history instead of a single message. The agent will use that history as context for the next reply. Remember, though, that this history is only what we send in this call – we will add persistent memory in the next section.

Finally, the demo shows streaming:

```
for token, metadata in agent.stream(
    {"messages": [("human", "Tell me about Luna City")]},
    stream_mode="messages",
):
```

```
    if token.content:
        print(token.content, end="", flush=True)
```

Streaming returns tokens as they are produced instead of waiting for the complete response. That is what makes a chat interface feel responsive.

Demo code: `src/01-foundation/creating-agent.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

6.4. System Prompt

The system prompt is the instruction the model receives before the conversation starts. It sets the role, the rules, and the expected shape of the answer.

```
system_prompt = "You are a science fiction writer, create a capital city at
the user request."

agent = create_agent(
    model=model,
    system_prompt=system_prompt,
)
```

Compare a run with the system prompt to a run without it. The user question is the same, but the answers differ because the model is playing a different role.

A system prompt is also a good place to put examples.

In the few-shot example, we show the model a few input/output pairs:

```
User: What is the capital of mars?
Scifi Writer: Marsialis

User: What is the capital of Venus?
Scifi Writer: Venusovia
```

The model recognizes the pattern and follows it for the new question. This is useful when the desired behavior is easier to demonstrate than to describe.

We can make the output more consistent by describing a structure:

```
Name: The name of capital city
Location: Where it is based
Vibe: 2-3 words to describe its vibe
Economy: Main industries
```

But a prompt is still just text. If the application needs a predictable object, define a schema instead:

```
from pydantic import BaseModel

class CapitalInfo(BaseModel):
    name: str
    location: str
    vibe: str
    economy: str

agent = create_agent(
```

```
    model=model,
    system_prompt=system_prompt,
    response_format=CapitalInfo,
)
```

Now the result is available as `response["structured_response"]`, and the fields are validated by Pydantic. This is much safer than parsing free-form text in application code.

Demo code: `src/01-foundation/system-prompt.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

7. Tools

Purpose: Give the agent the ability to act, not just talk.

Until now, our agent could only produce text. It could reason, but it could not *do* anything outside the conversation.

A tool is a function that we expose to the model. The model does not run Python by itself. We describe the tool to it – its name, what it does, and what arguments it accepts. The model then returns a tool call, our code executes the function, and the result is sent back to the model as a tool message. This loop is what makes an application agentic.

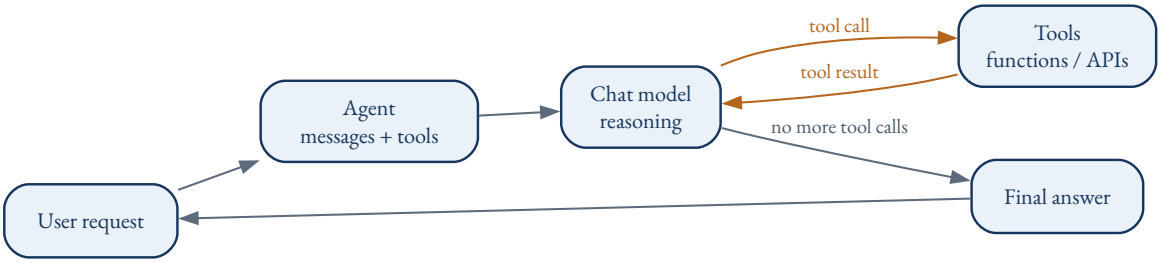


Figure 2: The tool-calling loop: the model requests a tool, our code runs it, and the result returns to the model.

If you want to go deeper: LangChain docs – Tools

7.1. Defining a tool

The `@tool` decorator from `langchain.tools` turns a normal Python function into a tool.

```
from langchain.tools import tool

@tool
def square_root(x: float) -> float:
    """Calculate the square root of a number."""
    return x ** 0.5
```

The function name, the type hints, and the docstring are not decoration. LangChain uses them to build the schema that the model sees. A vague docstring gives the model a vague idea of when to use the tool, so write it as if you are explaining the tool to a new teammate.

There are a few equivalent ways to define the same tool:

- `@tool` – uses the function name and docstring.

- `@tool("square_root")` – overrides the tool name.
- `@tool("square_root", description="Calculate the square root of a number")` – overrides both.

A tool can also be called directly, without an agent:

```
tool1.invoke({"x": 49})
```

This is useful for testing the function and seeing the exact input schema.

7.2. Giving tools to an agent

Once the tool is defined, pass it to the agent through the `tools` parameter.

```
agent = create_agent(
    model=model,
    tools=[square_root],
    system_prompt="You are an arithmetic wizard. Use your tools to calculate
the square root of any number.",
)
```

Ask the agent *What is the square root of 49?* and then inspect the response messages. You will see more than one message: the model first requests the tool, our code runs it, and then the model uses the result to answer. The tool call itself is available in `response["messages"][1].tool_calls`.

This is a good moment to look at the whole message list instead of only the final answer. Seeing the intermediate steps makes the loop much easier to understand.

Demo code: `src/02-tools/tools.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

8. Short-term Memory

Purpose: Give an agent memory within a conversation.

Every call to an agent starts from the messages we pass in. If we do not store anything between calls, the agent has no idea what happened before.

8.1. No memory

Predict what will happen in `no-memory.ipynb`.

We tell the agent our name and favourite colour in one call. Then, in a separate call, we ask what our favourite colour is. Since the second call only contains the new question, the agent has nothing to remember. Any answer it gives is a guess, not a memory.

Demo code: `src/03-short-term-memory/no-memory.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

8.2. Threads and checkpointers

To remember a conversation, we need two things:

- A *thread* – an identifier for the conversation, passed as `thread_id`.
- A *checkpointer* – the component that saves the agent state after every step.

With both in place, each new call loads the saved state for that thread before continuing.

```python
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model,
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "1"}}
```

Now the first call can introduce us, and the second call – using the same config – can answer questions about what we said earlier.

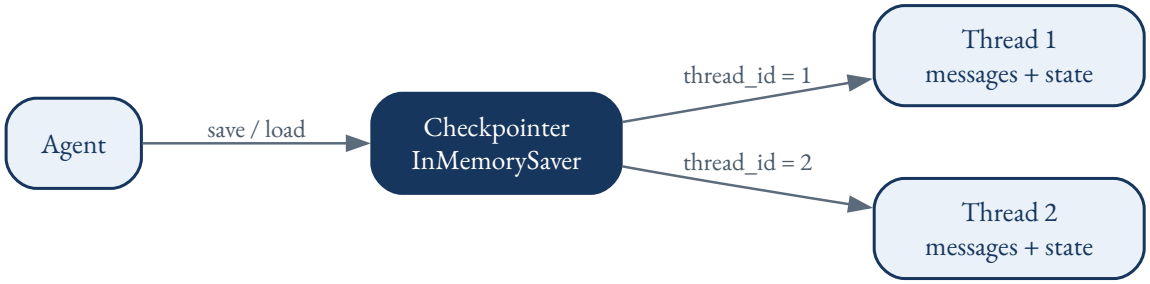


Figure 3: The checkpointer keeps a separate state for each `thread_id`.

`InMemorySaver` stores everything in memory, so it disappears when the process ends. That is perfect for a demo. For a real application, use a persistent checkpointer such as `SqliteSaver` or `PostgresSaver`.

Try changing the `thread_id` and observe how the agent's memory changes.

Demo code: `src/03-short-term-memory/checkpointer.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

Further reading: LangChain docs – Short-term memory

9. Managing Long Conversations

Purpose: Keep conversations useful as they grow.

Every message, tool call, and tool result consumes tokens. A model has a finite context window, and a long-running conversation will eventually reach it.

We have two common strategies:

- *Summarization* – compress older messages into a short summary.
- *Trimming* – remove messages that are no longer needed.

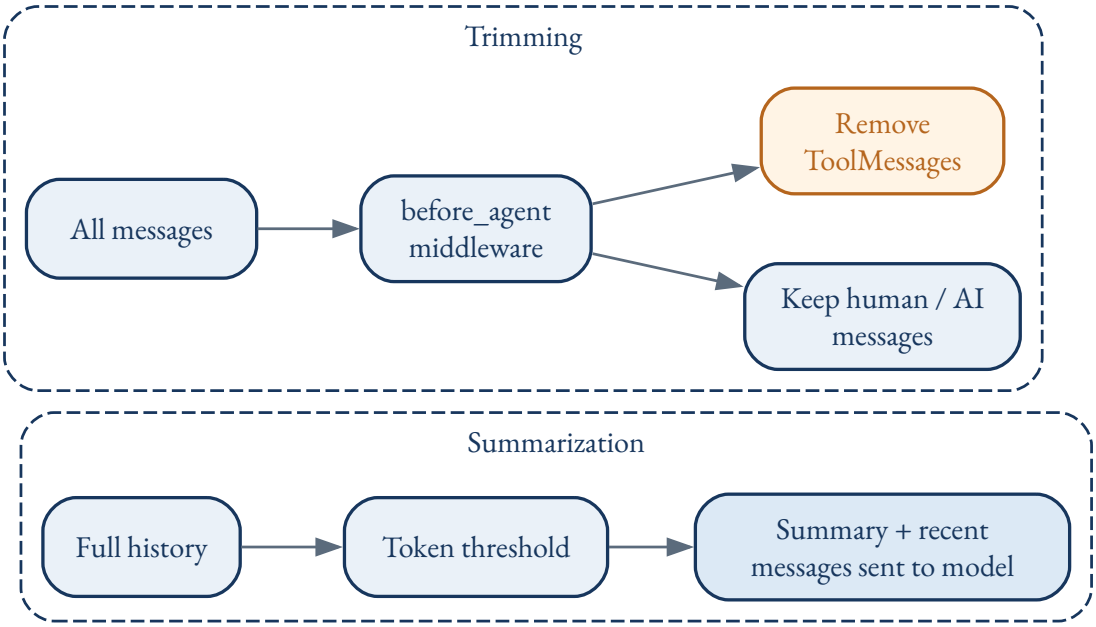


Figure 4: Two ways to keep a conversation inside the context window.

9.1. Summarization

LangChain provides `SummarizationMiddleware`. When the conversation crosses a trigger, the middleware asks a model to summarize the older part of the conversation, then keeps the most recent messages.

```
from langchain.agents.middleware import SummarizationMiddleware

middleware=[
    SummarizationMiddleware(
        model=summarization_model,
        trigger=("tokens", 100),
        keep=("messages", 1),
    ),
]
```

The values `100` and `1` are intentionally small so the effect is easy to see in a demo. In a real application they should match the model's context window and your tolerance for losing detail.

After the run, `response["messages"][0].content` contains the summary. The original conversation has been replaced in the model's context by that summary plus the messages we chose to keep.

Demo code: `src/04-managing-conversation/summarization.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

9.2. Trimming messages

Summarization is not the only option. Sometimes we know exactly which messages are disposable.

In the demo we write a small middleware using `@before_agent`. It runs before the agent starts, finds every `ToolMessage` in the state, and removes them with `RemoveMessage`.

```python
@before_agent
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, any] |
None:
    messages = state["messages"]
    tool_messages = [m for m in messages if isinstance(m, ToolMessage)]
    return {"messages": [RemoveMessage(id=m.id) for m in tool_messages]}
```

Tool messages are often large and are usually needed only while the model is deciding what to do next. Removing them can free a lot of context.

Be careful in a real agent: if you remove a tool result, keep the conversation coherent. The demo removes all of them so the behavior is obvious.

Demo code: `src/04-managing-conversation/trim_messages.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

Further reading: LangChain docs – Middleware

10. Deep Agents

Purpose: Move from a single agent loop to a harness built for long-running work.

`create_agent` gives us the core loop: the model, the tools, and the messages. That is enough for many tasks, but complex work needs more scaffolding.

Deep Agents is an *agent harness* built on top of LangChain and LangGraph. It keeps the same tool-calling loop, but adds capabilities that make an agent reliable over longer runs:

- Task planning and a to-do list.
- A virtual filesystem for reading and writing files.
- Context management: summarization, offloading, skills, and memory.
- Delegation to subagents with isolated context.
- Human-in-the-loop approval for sensitive actions.
- An execution environment with shell and code interpreter support.

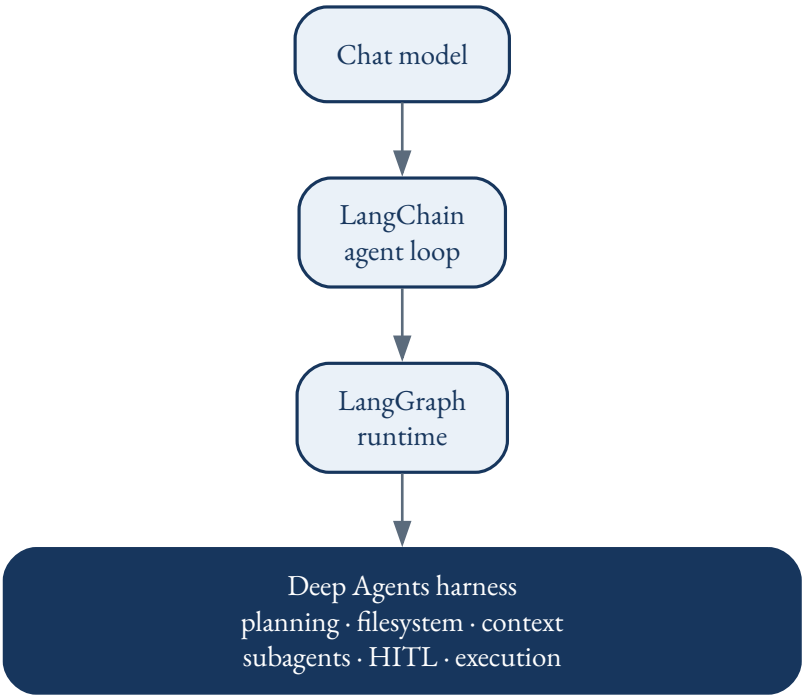


Figure 5: Deep Agents adds a harness on top of the LangChain loop and the LangGraph runtime.

The model is still the brain. The harness is everything around it that lets the agent work for more than one turn.

10.1. Running a Deep Agent

Creating one is deliberately familiar.

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
result = agent.invoke({"messages": [{"role": "user", "content": "What is an
LLM?"}]})
print(result["messages"][-1].text)
```

Try the same prompt with `create_agent` and with `create_deep_agent`. The answers may look similar, but the deep agent now has filesystem tools, planning, and the rest of the harness available behind the scenes.

Demo code: `src/05-creating-deepagents/creating_deep_agent.py`
Running: `uv run python src/05-creating-deepagents/creating_deep_agent.py`

Further reading: Deep Agents overview

11. Model Context Protocol (MCP)

Purpose: Connect an agent to external tools through one shared protocol.

The Model Context Protocol is an open standard for exposing tools, resources, and prompts to AI applications. Instead of writing a custom integration for every service, an application can connect to an MCP server and discover what it offers.

The LangChain documentation itself exposes an MCP server at docs.langchain.com/mcp. Our demo connects to it, lists the available tools, and lets the agent use one of them.

```
from langchain.mcp import MCPAdapter

async with MCPAdapter("https://docs.langchain.com/mcp") as adapter:
    tools = await adapter.list_tools()
```

Two ideas are worth noticing here.

First, *discovery*: the client asks the server which tools exist, and the server describes them. Second, *selection*: we do not have to give the agent every tool we discover. In the demo we filter the list down to a single allowed tool before creating the agent.

That is a useful habit. A remote server may expose many tools, and an agent should usually receive only the ones it needs for the task.

Notice also that MCP calls are asynchronous, so the demo uses `async/await` and `agent.ainvoke`.



Figure 6: MCP separates tool discovery from tool selection.

Demo code: `src/06-mcp/mcp-demo.py`
Running: `uv run python src/06-mcp/mcp-demo.py`

Further reading:
- Model Context Protocol
- LangChain docs – MCP

12. Human-in-the-Loop

Purpose: Let a human approve or change an action before it happens.

An agent that can act can also make mistakes with real consequences. It might send an email to the wrong person, delete a file, or call an API that costs money.

Human-in-the-loop gives us a checkpoint before those actions. In Deep Agents we enable it with `interrupt_on`.

```
agent = create_deep_agent(
    model=model,
    tools=[send_email],
    interrupt_on={"send_email": True},
    checkpointer=MemorySaver(),
)
```

The checkpointer is required: when the agent pauses, its state has to be saved so it can continue later.

The run stops before `send_email` executes. The result contains the pending action requests, and we decide what to do with each one.

The `version="v2"` argument makes the result easier to work with: `result.value` holds the agent state and `result.interrupts` holds anything waiting for a human. Use the same version when you resume, so the result shape stays consistent.

- *Approve* – run the tool call as proposed.
- *Edit* – change the arguments before running it.
- *Reject* – do not run it, and tell the agent why.

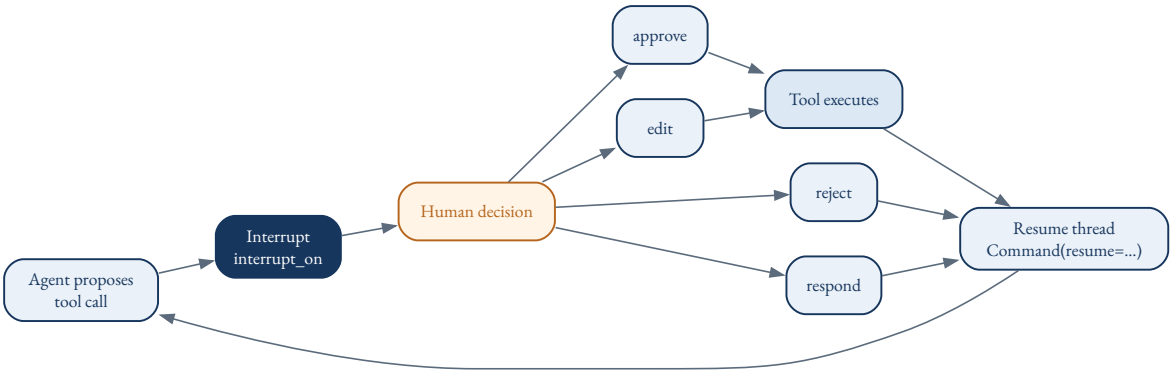


Figure 7: The agent pauses before the tool runs; the human decision determines what happens next.

After collecting the decisions, we resume the same thread:

```
result = agent.invoke(
    Command(resume={"decisions": decisions}),
    config=config,
    version="v2",
)
```

12.1. Approving, editing and rejecting

Run `hitl.py` and try all three decisions.

When you edit an email, notice that the tool result quotes the edited subject and body – not the original request. That is why the system prompt tells the agent to trust the tool result instead of assuming the request was carried out.

Demo code: `src/07-HITL/hitl.py`
Running: `uv run python src/07-HITL/hitl.py`

12.2. Responding to the agent

Sometimes the agent does not need permission. It needs information that only the user can provide.

In `hitl_response.py` we add an `ask_user` tool with a fixed set of choices. Its interrupt is configured with `allowed_decisions: ["respond"]`, so instead of approve/edit/reject, the human sends an answer back to the agent.

The email tool keeps its usual `["approve", "edit", "reject"]` decisions. This shows that different tools can have different levels of human control in the same agent.

The demo uses `questionary` to make the choices interactive.

Demo code: `src/07-HITL/hitl_response.py`
Running: `uv run python src/07-HITL/hitl_response.py`

Further reading: Deep Agents docs – Human-in-the-loop

13. The Deep Agent Environment

Purpose: Understand where a deep agent acts.

An agent is only as useful as what it can touch. The execution environment is the part of the harness that lets it take action.

Deep Agents describes four layers:

Layer	What it gives the agent
Tools	Custom functions, APIs, databases, and MCP servers
Virtual filesystem	Read, write, edit, search, and delete files through built-in tools
Filesystem permissions	Declarative rules for which paths may be read or written
Code execution	Shell commands through a sandbox, or JavaScript through an interpreter

The filesystem is *virtual*: the agent works with paths such as `/reference/chinook-sales.md`, and a backend maps those paths to real storage. This is a powerful abstraction, because the same agent can run against in-memory state, local disk, a database, or a remote sandbox without changing its tools.

13.1. Filesystem Backends

Purpose: Choose where the agent's files actually live.

A backend implements the storage behind the virtual filesystem. The demos use three of them:

- `StateBackend` – stores files in the agent state. Fast and simple, but tied to the thread.
- `FilesystemBackend` – maps paths to a real directory on disk.
- `StoreBackend` – stores files in a LangGraph store, which can persist across threads.
- `CompositeBackend` – sends different path prefixes to different backends.

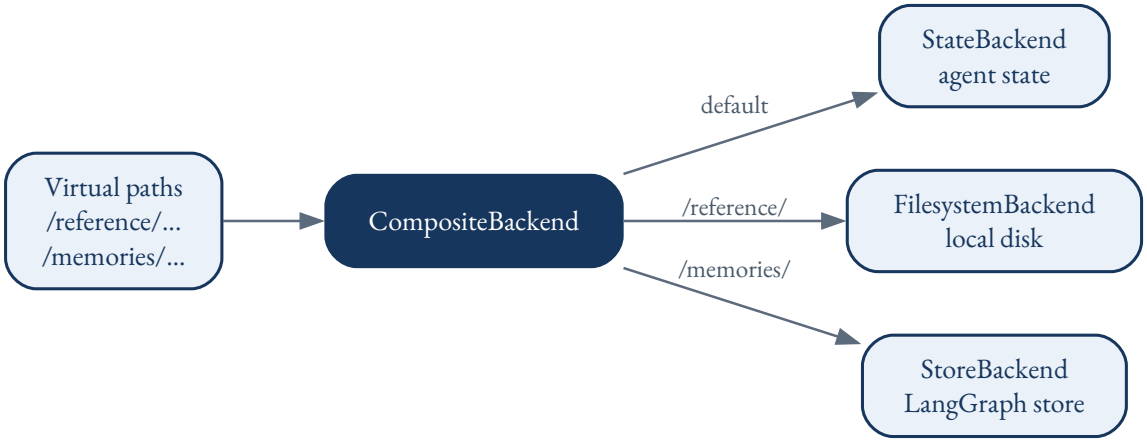


Figure 8: A composite backend routes each virtual path to the backend that should store it.

In `filesystem.py`, most paths use the default `StateBackend`, but anything under `/reference/` is routed to a real directory:

```
backend=CompositeBackend(
    default=StateBackend(),
    routes={
        "/reference/": FilesystemBackend(
            root_dir=str(reference_dir),
            virtual_mode=True,
        ),
    },
),
```

`virtual_mode=True` keeps the agent's view contained inside `root_dir`. It cannot use `..` to wander out of the directory we gave it.

The demo also adds a permission rule that allows writes under `/reference/**`. Permissions are evaluated in order, and the first matching rule wins. If no rule matches, the operation is allowed by default – so it is worth being explicit about what should be denied.

Demo code: `src/08-deepagent-environment/filesystem.py`
Running: `uv run python src/08-deepagent-environment/filesystem.py`

Further reading:
- Deep Agents docs – Backends
- Deep Agents docs – Permissions

13.2. Sandboxes and LocalShell

Purpose: Let the agent run commands.

Some tasks need more than file edits. They need a shell.

A sandbox backend adds an `execute` tool that runs commands in an isolated environment. `LocalShellBackend` is the local version: it runs commands on your machine, inside a chosen workspace directory.

```
from deepagents.backends.local_shell import LocalShellBackend

backend = LocalShellBackend(root_dir=workspace_dir, virtual_mode=True)
```

In the demo, the agent writes the first 15 Fibonacci numbers to `fib.py` and then runs it.

`LocalShellBackend` is excellent for learning and for trusted local workflows. It is not a security boundary. If you are running untrusted code, use a real sandbox provider instead.

Demo code: `src/08-deepagent-environment/localshell.py`
Running: `uv run python src/08-deepagent-environment/localshell.py`

Further reading: Deep Agents docs – Sandboxes

13.3. Interpreter

Purpose: Give the agent a programmable layer for multi-step data work.

A shell is not always the right tool. Sometimes we want a small, safe runtime where the agent can loop, transform data, and call tools programmatically.

`CodeInterpreterMiddleware` adds an `eval` tool that runs JavaScript in a QuickJS runtime. It has no shell access, no package installation, and no filesystem or network access.

The interesting part is *Programmatic Tool Calling*.

```
middleware=[CodeInterpreterMiddleware(ptc=["query_chinook"])]
```

With `ptc`, the tool is exposed inside the interpreter as `tools.queryChinook()`. The agent can write one JavaScript snippet that calls the tool several times, keeps intermediate values in variables, and returns only the final result.

Why does this matter? Consider a chain of dependent database queries: the second query needs the result of the first, and the third needs the result of the second. Without the interpreter, every intermediate result travels back to the model as a message. With the interpreter, those values stay in JavaScript and never enter the model's context.

That means fewer round trips and far fewer tokens.

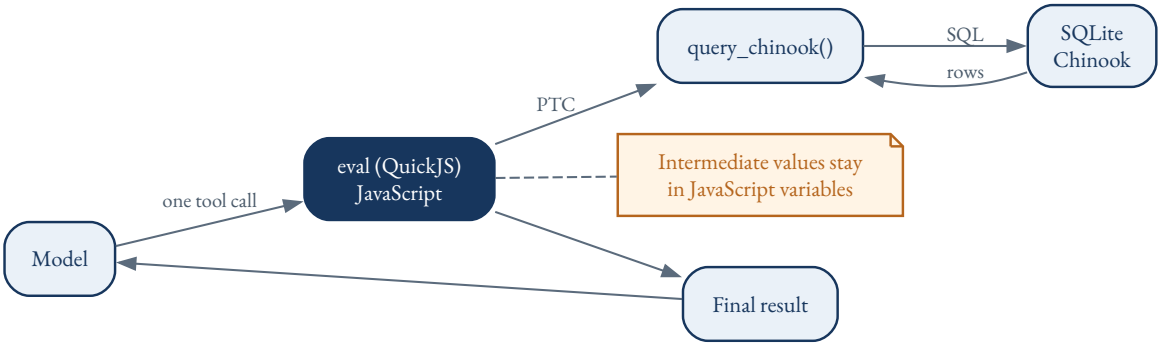


Figure 9: Programmatic tool calling keeps intermediate results inside the interpreter instead of sending them back to the model.

The demo runs the same analytics task twice – once with the interpreter and once without – against the Chinook database. Compare both the answers and the amount of work the model had to do.

Demo code: `src/08-deepagent-environment/interpreter.py`
Running: `uv run python src/08-deepagent-environment/interpreter.py`

Further reading: Deep Agents docs – Interpreters

14. Context Management

Purpose: Keep a long-running agent useful as its context grows.

A deep agent may work for hundreds of steps. The context window does not grow with it.

Context management decides what enters the model's context, what stays available outside it, and what can be forgotten.

There are four layers:

- *Input context* – the system prompt, memory, skills, and tool descriptions the agent starts with.

- *Compression* – summarization and offloading of older history and large results.
- *Isolation* – subagents that do heavy work in a separate context and return only a report.
- *Long-term memory* – persistent files that survive across threads.

14.1. Summarization and Offload

Purpose: See how a deep agent keeps working when the conversation gets long.

Summarization compresses older messages. Offloading moves large results out of the conversation and into the filesystem, leaving a reference behind.

The demo makes this visible by shrinking the model's reported context window:

```
model.profile = {**model.profile, "max_input_tokens": 6200}
```

With a smaller window, summarization happens quickly and we can watch it in a short run.

After each turn, the demo prints two things:

- how many messages are stored in the thread.
- what the model actually saw – either the full history, or a summary plus `messages[cutoff:]`.

The stored history is never trimmed. The agent keeps the raw conversation in state, while the model receives a compressed view.

The `_summarization_event` in state records where the cutoff happened.

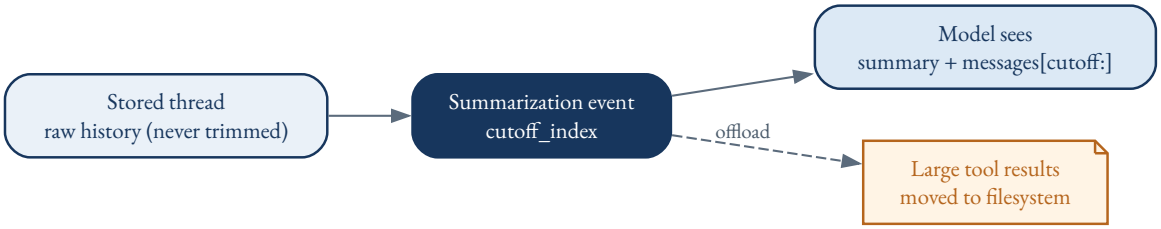


Figure 10: The full history stays in state; the model receives a compressed view.

Demo code: `src/09-summarization/summarization.py`
Running: `uv run python src/09-summarization/summarization.py`

Further reading: Deep Agents docs – Context engineering

14.2. Skills

Purpose: Package specialist instructions the agent loads only when needed.

A skill is a directory containing a `SKILL.md` file. The file starts with metadata and then contains instructions:

```
---
name: qualify-lead
description: Use when the user wants to qualify a sales lead or prospect.
---

# Qualify a Lead

Use the BANT framework...
```

Skills follow the Agent Skills standard. They can also include scripts, templates, and reference documents.

Deep Agents load skills with *progressive disclosure*. At startup, the agent sees only the name and description of each skill. When a task matches, it reads the full instructions.

This keeps the starting context small while still making detailed workflows available on demand.

```
agent = create_deep_agent(
    model=model,
    name="Sales_Assistant",
    backend=backend,
    skills=["/skills"],
    system_prompt="You are a sales assistant.",
)
```

The demo includes two skills: `qualify-lead` and `draft-pitch`. Ask the agent to qualify a lead, and watch it discover and load the right skill.

Demo code: `src/09-summarization/skills.py`
Running: `uv run python src/09-summarization/skills.py`

Further reading: Deep Agents docs – Skills

14.3. Memory

Purpose: Keep knowledge across conversations.

Skills are loaded on demand. Memory is always present.

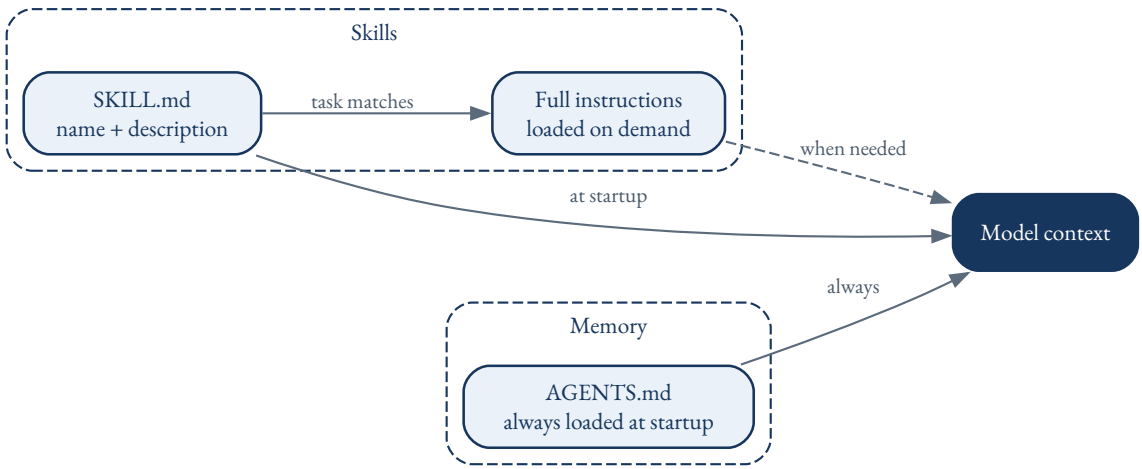


Figure 11: Skills enter context when a task needs them; memory is loaded at startup.

Memory uses `AGENTS.md` files. The content is loaded at startup and stays in the system context, so the agent follows it without being reminded.

```
agent = create_deep_agent(
    model=model,
    name="Memory_Agent",
    backend=CompositeBackend(
        default=StateBackend(),
```

```
        routes={"/memories/": StoreBackend(namespace=lambda _:
default_namespace)},
    ),
    store=store,
    memory=["/memories/AGENTS.md"],
    system_prompt="You are a helpful coding assistant for this project.",
)
```

Because the file lives in a `StoreBackend`, it can persist beyond a single thread. The demo seeds an `AGENTS.md` with project guidelines, then does two things:

- asks a question whose answer is in memory.
- asks the agent to update memory with a new team decision.

At the end, it prints the stored `AGENTS.md` so we can see the change.

That last step is important. Memory is not only read at startup; the agent can also write to it, so preferences and conventions carry forward.

Demo code: `src/09-summarization/memory.py`
Running: `uv run python src/09-summarization/memory.py`

Further reading: Deep Agents docs – Memory

15. Wrapping Up

We started with a simple question: can AI do more than answer?

Along the way we built up the pieces of an agentic application:

- a model as the reasoning engine,
- tools that let it act,
- memory and checkpoints that let it continue a conversation,
- middleware that keeps long conversations manageable,
- a deep agent harness for long-running work,
- MCP for external tools,
- human approval for sensitive actions,
- filesystems, shells, and interpreters for execution,
- and context management through summarization, skills, and memory.

The same ideas scale from a notebook demo to a production application. The harness does not replace the model – it gives the model a place to work.

15.1. Where to go next

The demos are adapted from two LangChain Academy courses:

- `src/01–04`: Introduction to LangChain – Python
- `src/05–09`: Introduction to Deep Agents

The courses cover more than we use here. We kept the parts that fit this session and left out topics such as multimodal messages, multi-agent systems, and deployment. If you want to continue, the next topics are delegation and subagents, LangSmith tracing, evaluation, and deployment.

Try changing the demos. Swap the model. Add a tool. Break something and read the trace. That is where the learning happens.